

重要なシステムサービス(主題1.09)

システム時刻の管理

1. 这道题考查的是网络时间协议（NTP）中非常著名的「NTPサーバプール（pool.ntp.org）」集群的底层工作机制。

在实际生产环境配服务器时，我们经常会无脑敲入 pool.ntp.org。但这并不是某一台超级服务器的名字，而是一个巨大的**分布式服务器池**。我们用大白话来看看当你去请求它时，后台发生了什么好玩的事。

💡 知识点总结

- pool.ntp.org 是一个由全球志愿者提供的、成千上万台 NTP 服务器组成的**虚拟巨型集群**。
- **DNS 轮询机制（ラウンドロビン）**：当你尝试解析 pool.ntp.org 时，它的 DNS 服务器并不知道你想连哪一台，为了**公平分摊流量（負荷分散 / 负载均衡）**，它会**每次都随机挑几台不同的 NTP 服务器 IP 地址**扔回给你。

✅ 正确答案

リクエストごとにランダムなNTPサーバのIPアドレスを返す

🔍 选项解析与详细说明

我们用一个非常形象的“前台派发小纸条”的过程，来看看你每一次去戳 pool.ntp.org 时的真实场景：

- a. **你发起请求**：服务器 A 尝试配置时间，向 pool.ntp.org 发起 DNS 解析请求。
- b. **DNS 前台盲盒派发**：pool.ntp.org 的官方 DNS 收到请求后，从它密密麻麻的服务器大名单里，**随便抓了 3 个 IP 地址**（比如 192.168.1.1 等）丢给服务器 A。
- c. **另一个人再发起请求**：一秒钟后，服务器 B 也发起了请求。DNS 前台再次摇号，**又随机抓了另外 3 个完全不同的 IP 地址**丢给服务器 B。
- d. **核心本质**：通过这种**每次リクエスト都返回随机 IP**的机制，全球海量的并发请求被均匀地分流到了几千台志愿者的服务器上，防止任何一台服务器被瞬间冲垮。所以第四项完全正确。

❌ 为什么其他选项是错的？

- 「**地理的に最も近い位置にあるNTPサーバの…**」：这是一个极其精妙的高级陷阱！如果题目写的是 jp.pool.ntp.org（带了国家前缀），它确实会优先返回日本境内的服务器 IP；但题目给的是不带任何前缀的总域名 pool.ntp.org，总域名是完全不管你在哪里的，只会面向全球大池子随机盲盒派发。

- 「登録されているすべてのNTPサーバの一覧を返す」：错。全球有几千几万台 NTP 志愿服务器，如果一次性把几万个 IP 列表全部吐给你，你的网络包直接就爆炸了。它每次只会随机挑 3 到 4 个 IP 返回。
 - 「時刻の同期を直接実行する」：错。 `pool.ntp.org` 本身只是一个虚拟的域名（DNS导航站），它自己不提供时钟信号，它只负责告诉你去连哪个具体的 IP，真正的同步是你的服务器和拿到的那个具体 IP 之间进行的。
2. NTPサーバと継続的に同期を行い、システムクロックのズレを少しずつ段階的に修正するプログラムは`ntpd`
3. 「/etc/ntp.conf」ファイルにおいて、NTPサーバを正しく指定しているものは`server 0.centos.pool.ntp.org`

 考场高频： `ntp.conf` 核心配置三大指令排查表

在 LPIC-1 考试中，关于 `/etc/ntp.conf` 文件的考查，只要抠住这三个高频关键字的职能即可：

配置文件关键字	 考场核心判定法（大白话）	 它的具体用途示例
<code>server</code> （本题）	 找大哥！指定要去同步的上游 NTP 服务器。	<code>server 0.centos.pool.ntp.org</code>
<code>restrict</code>	设防线！限制哪些客户端可以来连我、怎么连。	<code>restrict 192.168.1.0 mask 255.255.255.0 nomodi</code>
<code>driftfile</code>	记误差！指定用来记录本地时钟频率误差的文件。	<code>driftfile /var/lib/ntp/drift</code>

标准格式为： `server [NTPサーバのホスト名またはIPアドレス]` 。

 考试小秘籍

- a. “不画蛇添足”原则：Linux 里的很多经典服务（如 NTP、DNS等），在自己的配置文件里引用外界服务器时，关键字通通都是最原始的 `server`。出题人特别喜欢瞎编像 `ntpserver`、`dnsserver` 这种看似非常专业、实则根本不存在的拼写来钓鱼，考场上看到这种复合词直接划掉。
- b. 联动 pool 机制：你看这道题里的服务器地址写的是 `0.centos.pool.ntp.org`。结合我们前两道题学的知识，因为里面带了 `pool`，这就意味着当 `ntpd` 去连 `server 0.centos.pool.ntp.org` 时，后台的 DNS 每次都会随机返回不同的 IP 给它来实现负载均衡（负荷分散）！知识点是不是完美串联起来了？

4. 知识点总结

在 Linux 网络服务中，有一种统治了数十年的核心架构叫做 **C/S 架构（クライアント / サーバ——客户端 / 服务器架构）**。

- **核心考点**：既然我们的这台机器现在扮演的是“客户端（Client）”，要去向外部那个“信任的 NTP 服务器”索要时间，那么在配置文件里，我们就必须明确地指明对方的身份——“它（对方）是我的服务器（`server`）”。

- 因为这个配置文件本身就叫 `ntp.conf`，所以系统默认里面所有的配置都是服务于 NTP 协议的。在里面再写 `ntp`，就属于无效的重复（画蛇添足）。

✅ 正确答案

server

🔍 选项解析与详细说明：为什么 `ntp` 反而是错的？

我们来看为什么出题人故意放了 `ntp` 这个超级诱饵，以及其他选项为什么不符合 Linux 的脾气：

- 为什么不选 `ntp`？假如关键字叫 `ntp`，那配置就会写成：`ntp 0.centos.pool.ntp.org`。你想想，系统在读取 `ntp.conf` 时，已经知道这是 NTP 的地盘了。如果每行开头还写 `ntp`，系统会觉得很呆。更重要的是，`ntp` 这个词没有体现出两台机器之间的“上下级角色关系”。
- 为什么必须选 `server`？Linux 讲究的是“角色定义”。不管是 NTP 时间同步、DNS 域名解析、还是 LDAP 认证，只要是“指定我要连哪个上游老大”，Linux 统一都用最纯正的英文单词 `server`。它告诉系统：“后面跟着的这个 IP/域名，是提供时间服务的 **サーバ（服务器）**”。

5. Linuxの動作するシステムで扱う時刻には、ハードウェアクロックとシステムクロックがあります。

種別	表示/設定 コマンド	内容
ハードウェアクロック	hwclock	コンピュータに内蔵された時刻。 電源OFFの状態でも動作
システムクロック	date	Linuxカーネルが持つ時刻。システム起動時に ハードウェアクロックの値が設定される。 システムのイベントの時刻として使用される

ハードウェアクロックの参照や設定を行うコマンドは**hwclock**

💡 知识点总结

在 Linux 电脑中，其实同时存在着两个完全独立的时间：

- a. **システムクロック（系统时钟）**：由 Linux 内核在系统启动后负责管理的**虚拟时钟**。平时我们用 `date` 查出来的就是它。
- b. **ハードウェアクロック（硬件时钟）**：由主板上那颗 RTC 晶振和纽扣电池供电的**物理时钟**。即便电脑关机、断电，它也会在后台默默地一秒一秒走动。

而专门负责“直接查看（参照）或者把时间写入（设定）到主板硬件”的命令，就是 `hwclock`（**Hardware Clock** 的缩写）。

🔍 选项解析与详细说明

我们用一张「主板硬件 **VS** 系统内核」的时间互抄图，来看看 `hwclock` 在日常运维中到底是怎么工作的：

- **核心参数一（读取）**：直接输入 `hwclock`，就是去读取主板芯片里现在跳到了几点几分。
- **核心参数二（系统同步到硬件）**：`hwclock -w`（或 `--systohc`）。大白话就是：系统时间（System）是准的，硬件时间（Hardware）走慢了，通过 `-w`（**Write，写入**）动作，把当前的系统时间**写入并覆盖**到硬件时钟里。
- **核心参数三（硬件同步到系统）**：`hwclock -s`（或 `--hctosys`）。大白话就是：电脑刚开机（Start）时，系统时钟是一片空白，需要执行 `-s`（**Set，设置**）动作，去把硬件时钟里的时间**读取并设置**给系统时钟。

✗ 为什么其他选项是错的？

- `date`：它只负责管操作系统内部的「**系统时钟（系统虚拟时间）**」，根本触碰不到主板底层的硬件芯片。
- `ntpdate`：我们在前两题刚学过，它是从**网络上的 NTP 服务器**获取时间并粗暴修改系统时间的工具。

能完美理解第一个选项（“获取并立即反映到系统时钟”），说明你已经抓住了 `ntpdate` 最核心的动作特征——“**粗暴的一次性校对**”。

至于剩下的一个正确答案，其实只要理清“**谁向谁要时间**”的上下级角色关系就能瞬间秒杀。另一个正确答案是：「**NTPクライアントである（是 NTP 客户端）**」。

在网络时间协议（NTP）的世界里，两台电脑通信时只有两种身份：

NTPサーバ（NTP 服务器）：手里握着正确时间，专门**施舍、提供**时间给别人的“大老板”。

NTPクライアント（NTP 客户端）：自己时间不准，主动跑去向服务器**索要、接收**时间的“小弟”。

`ntpdate` 是一个命令，当你敲下 `ntpdate pool.ntp.org` 时，是你的电脑主动去问别人要时间。因此，`ntpdate` 扮演的角色百分之百是一个 **NTP 客户端工具**。

✗ 为什么不选 `NTPサーバである`？

- **原因**：`ntpdate` 只能“向别人要时间”，它自己**没有**在后台开放端口、默默等待别人连进来向它要时间的功能。它不是服务器（Server），它只是一个纯粹的客户端（Client）校对工具。

✗ 为什么不选 `stratum 0~16の階層構造を持っている`？

- **原因**：这个叙述本身包含了一个**致命的内核级概念错误**。NTP 的阶层（Stratum）结构确实存在，但它的范围是 `1~15` 代表可用，而 `16` 代表“**处于未设定/不可用状态（同期不可）**”。`0` 则代表原子钟、GPS 等纯物理时钟源（人类服务器无法达到 Stratum 0）。因此“0~16的阶层结构”这个数值范围描述在 Linux 考试里是个高频的错误干扰项。
- LPIC-1 特别喜欢考 NTP 的数字。请把这个规则刻在DNA里——“**Stratum 15 是天花板（最底层），Stratum 16 就是死人（不可用）**”。选项里只要出现 `0~16` 或者说 `16层可以用` 的，通通都是出题人在胡编乱造，直接打叉。

❌ 为什么不选 `ntpd` が実行する？

- **原因**：这属于“同行相轻”。`ntpd` 和 `ntpddate` 是 Linux 里平级的两个独立程序。`ntpd` 是后台温柔对时的守护进程，`ntpddate` 是前台粗暴对时的命令，两者是互斥关系。甚至当 `ntpd` 在运行时，由于端口被占，你直接去敲 `ntpddate` 还会报错。所以 `ntpd` 绝对不可能去执行 `ntpddate`。
- `tzselect` / `tzconfig`：这两个命令是用来修改系统的「タイムゾーン（时区 / Time Zone）」（例如把时间切换到 `Asia/Tokyo` 或者是 `Asia/Shanghai`），它们不负责时钟本身的数值设定。

命令/程序名称	👤 在网络中扮演的角色身份	🔒 考场核心判定法（大白话）
<code>ntpddate</code>	🔒 NTP クライアント（客户端）	🔒 纯跑腿的小弟，主动去命令行屁股后面的网址要时间。
<code>ntpd</code> / <code>chronyd</code>	既是客户端，又是服务器	它们不仅能向上游老大要时间（做客户端），还能常驻后台等局域网其他小弟来向自己要时间（做服务器）。

📊 考场高频：时间管理两大核心命令决斗表

在 LPIC-1 考试中，出题人最喜欢拿这两个命令互相作为干扰项。只要分清“内核”与“硬件”即可：

核心管理命令	🔒 考场核心判定关键词	💡 它的管辖领地	经典操作命令
<code>hwclock</code>	ハードウェアクロック、RTC	🔒 主板电池里的那个纯物理时钟。	<code>hwclock -w</code> (写进主板)
<code>date</code>	システムクロック、表示・設定	Linux 系统跑在内存里的虚拟时间。	<code>date -s "12:00:00"</code>

联动上一题的反向控制：上一题我们问到：限制某个网段客户端不能来修改我这台 NTP 服务器时间（进行限制防线控制）应该用哪个关键字？答案就是 `restrict`（限制）。结合这一题，NTP 同步完系统时间（`date`）后，我们通常都会紧接着敲一句 `hwclock -w`，顺手把准的时间也写进主板里。这样即使服务器以后意外断电重启，时间也依然是精准的！

date コマンドでできることは

システムクロックを設定する

任意の形式で日付時刻を表示する

システムクロックを表示する

`date` 命令是「系统时钟（系统时间）」的绝对代言人。它只能用来查看系统时间、修改系统时间、或者把系统时间改变成各种你想要的格式来显示。它没有任何能力去触碰主板上的硬件芯片，也无法直接打印日历。

6. `hwclock` コマンド（ハードウェアクロックの時刻をシステムクロックに反映させるには「`hwclock`」コマンドを使用します。）

オプション	説明
-r	ハードウェアクロックを表示（オプション無しと同様）
-s --hctosys	ハードウェアクロックの時刻をシステムクロックに反映させる
-w --systohc	システムクロックの時刻をハードウェアクロックに反映させる

先明确 `hwclock` 命令的本职工作：它是 **Hardware Clock** 的缩写，专职负责**主板电池硬件时钟**与**系统内存虚拟时钟**之间的“抄写、对齐”工作。

- `-r`：直接读取显示硬件时间。
- `-s` (`--hctosys`)：把硬件时间复制给系统时间。
- `-w` (`--systohc`)：把系统时间复制给硬件时间。

短选项 (`-r`、`-s`、`-w`) 的荒诞大白话谐音

`-r` → 谐音：“肉” 🍖 (Read)

- 荒诞画面：你用眼镜去看一块**“肉 (`-r`) ”**。
- 记忆链接：`-r` 就是 Read，不作任何修改，只是干巴巴地**看 (表示) **硬件时钟。

`-s` → 谐音：“死” 💀 (Sys)

- 荒诞画面：主板硬件时钟太强大了，一巴掌把系统时钟给拍**“死 (`-s`) ”**了，系统只能顺从地变成硬件的时间。
- 记忆链接：`-s` 改变的是 System (系统时钟)，即 **硬件** → **系统**。

`-w` → 谐音：“喂” 🍼 (Write)

- 荒诞画面：系统时钟长大了，拿着奶瓶去**“喂 (`-w`) ”**那个嗷嗷待哺的硬件老顽童。
- 记忆链接：`-w` 就是 Write (写入)，把系统的精准时间**写入 (喂给) 硬件**。

长选项 (`--hctosys`、`--systohc`) 的拆字拼音绝招

这两个长选项长得极其吓人，其实它们是由 **3个缩写** 加上英文单词 `to` (**变成/给**) 连起来的拼音游戏：

- `hc` = **Hardware Clock** (硬件时钟)
- `sys` = **System Clock** (系统时钟)

`--hctosys` → 拆解：`hc` + `to` + `sys`

- 无厘头读法：“火 (h) 车 (c) 土 (to) 司 (sys)” 铺在系统上！
- 逻辑方向：**Hardware Clock** → **TO** → **System** (硬件时间传给系统)。

`--systohc` → 拆解：`sys` + `to` + `hc`

- 无厘头读法：系统上长出了“系统 (sys) 土 (to) 豪 (h) 词 (c)” 刻在主板上！
- 逻辑方向：**System** → **TO** → **Hardware Clock** (系统时间传给硬件)。

7. `timedatectl` コマンドで、NTP による時刻同期を有効・無効にするサブコマンドは `set-ntp`

`timedatectl` コマンドは、`systemd` の動作するシステムにおいてシステム時刻の表示や設定を行うコマンドです。

書式: <code>timedatectl</code> [サブコマンド]	
サブコマンド	説明
<code>status</code>	現在の日付時刻や状態の表示(デフォルト)
<code>set-ntp yes no</code>	NTP による時刻同期の有効 無効
<code>set-time</code> 日付 時刻 書式) <code>YYYY-MM-DD</code> (西暦-月-日) <code>hh:mm:ss</code> (時:分:秒) <code>YYYY-MM-DD hh:mm:ss</code>	日付や時刻の設定 時刻を指定しない場合は0時0分0秒になる
<code>set-timezone</code> タイムゾーン	タイムゾーンの設定
<code>list-timezones</code>	タイムゾーンの一覧表示

NTP (Network Time Protocol) による時刻同期はデフォルトでは有効になっています。

例; `timedatectl set-time "2019-05-01 15:30:27"`

💡 知识点总结

在现代 Linux 中，修改系统配置的 `systemd` 工具链（如 `timedatectl`、`localectl` 等）非常讲究命名规范。

- **核心考点：**只要涉及到“修改、设置某项具体配置的开关”，子命令一律以 `set-` 开头。
- 专门用来控制 **NTP 网络同步开启或关闭** 的子命令就是 `set-ntp`。

其标准使用书式为：

- 开启 NTP 自动对时 ➡ `timedatectl set-ntp true` （或 `yes`）
- 关闭 NTP 自动对时 ➡ `timedatectl set-ntp false` （或 `no`）

`timedatectl` 的子命令设计非常有规律：

- **查状态：**直接用 `status`。
- **改设置：**统一用 `set-` 开头（后面接你要改的对象：时间、时区或 NTP）。
- **捞列表：**统一用 `list-` 开头（用来查系统支持的所有时区）。

`timedatectl` 全能通关口诀

💡 10秒硬核记忆口诀（建议截图保存）：

“时代控制（`timedatectl`）大管家，命令规律超级佳。” “单打独斗查状态（`status`），其余动作带尾巴。” “只要涉及改设置，前缀必须 `set-` 刷：” “改时间叫 `set-time`，开关网络 `set-ntp`。” “想要切到东京去，`set-timezone` 拿下他！” “几百时区怎么查？`list-timezones` 打印出全家！”

永久、全局生效。彻底把整台服务器的默认时区全改掉（需要 root 权限）。

8. “お互いに” 终极条件反射：在所有的 Linux 网络协议题目里（包括 NTP 和 Chrony），只要在题干中看到了 **お互いに同期**（互相同步）、**対等な関係**（对等关系）或者 **対称**（Symmetric）这类字眼，别看别的，选项里百分之百是在考 **peer** 这个单词。

Chrony（クローニー: ntpdに代わるNTPサーバ/クライアント）の設定ファイルは「/etc/chrony.conf」です。

「/etc/chrony.conf」の主な設定項目	
設定項目	説明
server	時刻を取得するNTPサーバを指定 サーバ/クライアントの関係
peer	時刻を取得するNTPサーバを指定 同じstratum(階層)同士
pool	時刻を取得するNTPサーバのプール (複数アドレスを集約した名前)を指定
driftfile	自身の時刻のズレ具合を記録する ファイルを指定
rtcsync	システムクロックをハードウェアクロック (RTC)にコピーする

“Chrony 配置（**chrony.conf**）有秩序，三个找人俩看地。”

“单向拜师认老大，老实敲上 **server** 记。”

“同层同辈（**stratum**）互对齐，认准伙伴是 **peer** 聚。”

“盲盒池子一整片，打包引入写 **pool** 里处理。”

“漂移误差（drift）存哪里？**driftfile** 记录器。”

“系统复制到硬件，实时同步 **rtcsync** 齐！”

我们结合口诀和考纲图片，逐一拆解这 5 个核心配置项的真实功能：

1. **server**（单向拜师认老大）

- **大白话**：指定上游 NTP 服务器。两台机器是严格的“主/从（サーバ/クライアント）”关系，本地机器单向跟老大对齐。
- **书式**：`server 0.centos.pool.ntp.org`

2. **peer**（同层互对齐）

- **大白话**：就是你上一道题刚做对的对等节点配置。用于相同层级（同じstratum階層同士）的服务器之间互相对齐、互相同步。
- **书式**：`peer 192.168.1.100`

3. **pool**（盲盒池子一整片）

- **大白话**：当你要引入一个由多个 IP 组成的服务器集群（域名包装出来的池子）时使用。

◦ 书式: `pool pool.ntp.org`

4. `driftfile` (漂移误差存哪里)

◦ 大白话: `drift` 在英文里是“漂移、偏差”的意思。因为本地时钟由于主板发热等原因每天都会有微小的走时误差, 这个配置项专门指定一个文件, 用来记录本地时钟的走时误差 (ズレ具合)。

◦ 书式: `driftfile /var/lib/chrony/drift`

5. `rtcsync` (实时同步到硬件)

◦ 大白话: `rtc` 就是我们学过的硬件时钟 (Real-Time Clock), `sync` 是同步。这个配置只要写在文件里, 内核就会每隔 11 分钟自动把精准的系统时钟 (系统时间) 复制/写入到硬件时钟里, 省去了我们手动敲 `hwclock -w` 的麻烦!

◦ 书式: 直接单写一行 `rtcsync`

 考场高频: Chrony 核心配置关键字精准判定表

在 LPIC-1 考试中, 只要抠住题干说明里的“核心日文词”, 就能一秒排除干扰项:

考场核心日文关键词	瞬间联想到的英文单词	✅ 唯一的正确配置项
サーバ/クライアントの関係	🔒 服务器 → Server	server
同じstratum (階層) 同士 / お互いに	🔒 同辈/对等 → Peer	peer
NTPサーバのプール	🔒 水池/集群 → Pool	pool
時刻のズレ具合を記録	🔒 漂移 → Drift	driftfile
システムクロックをハードウェアにコピー	🔒 硬件同步 → RTC + Sync	rtcsync

9. 在 Linux 中, 有一个专门用来控制系统读取哪里的时间配置的标准特殊环境变量, 那就是 `TZ` (Time Zone 的缩写)。无论是编写脚本还是临时在当前黑窗口切换时区, 只要使用 `export TZ=时区路径`, 系统就会立刻改变当前 shell 进程及其子进程的时间显示。

🔒 临时、局部生效。只改变当前窗口和当前用户的时区, 不影响整台服务器。

`ntpdate pool.ntp.org`

10. Chrony (クローニー) はntpdに代わるNTPサーバ/クライアントです。RHEL7やCentOS7以降では標準の時刻同期システムとなっています。Chronyのデーモンはchronydです。Chronyの設定ファイルは「/etc/chrony.conf」です。

在 Linux 操作系统中, 所有在后台默默无闻地常驻运行、为系统提供某种服务的程序, 它的官方术语叫做 守护进程 (日文: デーモン / 英文: **Daemon**)。Linux 开发者有一个极其统一的肌肉记忆——“守护进程的名字, 通常都是【服务软件名】的屁股后面强行加一个字母 `d` (代表 **Daemon**)”。

11. 在 Linux 电脑内部, 操作系统内核的表 (系统时钟 / System Clock) 在底层永远只跑一个绝对时间, 那就是 `UTC` (世界标准时间, 也可以理解为格林威治时间)。

- **核心本质**：Linux 系统自己不关心你在日本还是在中国，它在内存里数秒时永远按 UTC 数。
- **ローカルタイム（本地时间/系统时刻）** 的诞生：只有当人类想要看时间（比如敲下 `date` 命令）或者日志需要打印时间时，Linux 才会拿起 **タイムゾーン（时区）** 这把尺子，在 UTC 的基础上进行**加减法转换**，从而吐出符合人类当地习惯的本地时间。
- ✖️ **UTCを環境変数として設定するとローカルタイムを参照するようになる**
你原本的理解：“把世界标准时间（UTC）设为环境变量，就能让系统去看本地时间。”

为什么不对？

- i. **方向反了**：我们在前面刚学过，修改时区用的环境变量叫做 **TZ**（Time Zone），从来没有一个环境变量叫 **UTC**。
- ii. **逻辑反了**：系统变量 **TZ** 里面装的是诸如 `Asia/Tokyo` 这样的**时区外壳**，它的目的是把内核的 UTC 剥皮加工成肉眼可见的本地时间。我们不会把 UTC 当成变量设进去反向参考本地时间。所以第 3 项在语法和逻辑上都是完全说不通的。

12. 经典 NTP 服务（`ntpd`）的**专用管理/排查工具**—— `ntpq`（NTP Query）命令的核心选项功能。

在实际运维中，当你配置好了 `ntpd` 之后，服务到底有没有成功连上外网、对时状态正不正常，我们不能两眼一抹黑，必须使用 `ntpq` 去敲门问诊。

- **`ntpq` 命令的本质**：它是 **NTP Query**（查询）的缩写。它不仅可以单行直接输出结果，还支持进入一个**交互式的小黑窗口（对话モード / 交互模式）**。
- **`-i` オプション（核心考点）**：`-i` 对应英文单词 **Interactive（交互/对话）**。只要在启动时加上 `-i`，系统就会打开一个专属的提示符窗口（类似 `ntpq>`），允许管理员在里面连续输入各种查询子命令。
- **默认行为**：非常有趣的是，如果你直接在终端敲 `ntpq` 而不加任何参数，它**默认（デフォルト）**就会带上 `-i` 的效果，直接进入交互模式。

核心选项	📌 考场核心判定日文词	🧠 对应英文单词	💡 大白话一句话说明
<code>-i</code> （本题）	📌 対話モードで起動	📌 Interactive	📌 进入交互式黑窗口，变成 <code>ntpq></code> 让你在里面连续打字。
<code>-p</code>	同期状態を一覧表示	Peers	单行直出结果！ 噼里啪啦把所有上游服务器的对齐状态一次性

“NTP 状态不好说，快请 `ntpq` 来号脉。”

“想要单行查列表（一覧表示），认准 `-p`（Peers）不等待。”

“想要对话（対話モード）进窗口，加个 `-i`（Interactive）最合拍。”

“默认（デフォルト）也是 `-i` 进，考场遇到稳稳爱！”

看到 `-i` ➡️ 想 Interactive（交互/对话） ➡️ 选 `対話モード`。

看到 `-p` ➡️ 想 Peers（同辈/上游列表） ➡️ 选 `一覧表示`。这两个字母的逻辑是百分之百焊死的。

remote	refid	st	t	when	poll	reach	delay	offset	jitter
+li763-114.membe	157.7.154.23	3	u	59	64	377	26.792	1.132	2.265
+secure.opticner	223.255.185.2	2	u	54	64	377	17.111	0.313	2.712
+sv2.localdomain	103.1.106.69	2	u	53	64	377	23.466	-0.133	2.467
*60-56-214-78f2..GPS.		1	u	68	64	377	22.244	0.386	0.469

-p的结果

解读 `ntpq -p` 的结果，最核心的就是盯紧两样东西：最左侧的「状态符号（状态マーカー）」以及「st（Stratum阶层）列的数字」。

核心符号判定：

- *****（星号）：代表「現在の同期先」。这是最完美的符号，说明本台机器当前正在以它为基准进行时间同步。
- **+**（加号）：代表「同期可能（候補）」。说明它的时间也很靠谱，作为“备胎”随时准备在星号倒下时顶替上去。
- **无符号（空格）**：如果最左侧什么符号都没有，通常说明无法连接或没有响应。

核心数字判定（st列）：

- **st 16**：我们在前面的题目中强调过，**16 代表不可用/未同步（同期未完了）**。只要 st 是 16，说明这条线路绝对是处于断开、不可用状态的。
- “星号（*）高照是正宫，加号（+）紧跟是候选。”
- “星跟加号最靠谱，都在陪我‘时刻同步间（時刻同期している）’。”
- “减号（-）剔除不用管，看到 16（st 16）断了线（同期できない）！”

13. 「/etc/localtime」と「/etc/timezone」ファイルをまとめて設定できるコマンドは「**tzconfig**」です。

tzconfig 的本质：后缀 **config** 是 **Configuration**（配置）的缩写。它的权力非常大，是用来“动手修改”文件的。只要运行它，它就能同时把系统的全局时区配置文件（`/etc/localtime` 和 `/etc/timezone`）全部一起改掉。

14. 環境変数「TZ」や「/etc/timezone」ファイルで指定する値は「**tzselect**」コマンドで確認できます。

tzselect 的本质：后缀 **select** 是 **Selection**（挑选/查看）的缩写。它是一个温柔的“只读向导”。它**绝对不会自动去修改任何系统文件**。它只是通过向你提问（比如：你在哪个大洲？你在哪个国家？），帮你“确认/找出来”符合你要求的时区字符串（比如输出 `Asia/Tokyo`）。然后由你手动把这个字符串写进环境变量 `TZ` 或者 `/etc/timezone` 中。

15. システムで使用するタイムゾーンを指定するテキストファイルは「**/etc/timezone**」

/etc 目录：全称是 **Etcetera**，它是 Linux 系统中专门用来存放系统全局配置文件（設定ファイル）的最高大本营。

`timezone` 文件：名字就是“时区”的英文全拼，里面只写了一行非常干净的文本（例如 `Asia/Tokyo`），用来告诉整台电脑：“**全系统默认用这个时区**”。

16. `date` コマンドの書式は以下の通りです。

`date [MMDDhhmm[[CC]YY][.ss]]`

（MMは月、DDは日、hhは時、mmは分、CCは西暦の上2桁、YYは西暦の下2桁、ssは秒）

例子：`date 072019302010.20`

2010年 7月 20日 火曜日 19:30:20 JST

17. 在没有现代化命令的传统 Linux（或者编写自动化运维脚本时），修改整台服务器的**全局永久时区**，本质上就是做一次“文件覆盖”。

我们需要用到两个核心角色：

- a. `/usr/share/zoneinfo/`（**全球时区大仓库**）：这是一个大目录，里面分门别类地存放着全球几百个城市的时间规则二进制文件（比如 `Asia/Tokyo`、`Asia/Shanghai`）。它们是**只读的原材料（复制源）**。
- b. `/etc/localtime`（**当前生效的时区配置文件**）：这是整台电脑操作系统在计算时间时真正去读取的**最终配置文件（复制目的地）**。
 - **核心动作**：我们要把“东京”的原材料，**覆盖复制**到系统的最终配置文件里。
 - **标准语法**：`cp [大仓库里的具体城市文件] [系统的最终配置文件]`

18. `chronyc` コマンドで `chronyd`（`chrony` デーモン）の管理を行います。

書式: <code>chronyc</code> [サブコマンド]	
サブコマンド	説明
<code>activity</code>	オンライン/オフラインのNTPサーバ数を表示
<code>sources</code>	時刻のソースを表示
<code>sourcestats</code>	時刻のソースの統計情報を表示
<code>tracking</code>	トラッキング(時刻同期の詳細情報)を表示

现代化时间管理工具 `chronyc` 的常用子命令（サブコマンド）。正如我们前面聊过的，`chronyc`（末尾的 `c` 代表 Client）是人类管理员用来盘问、查看后台守护进程 `chronyd` 工作状态的前台控制台工具。

`chronyc` [子命令] 的标准职责划分：

- `activity`：看数量。显示目前在线/离线的 NTP 服务器**数量（数）**。
- `sources`：看名字。显示当前正在连接的时间**源列表（ソース）**。
- `sourcestats`：看报表。显示时间源的**统计信息（統計情報）**。
- `tracking`：看细节。显示当前时间同步的**详细跟踪信息（詳細情報）**。

核心子命令	🔒 考场核心判定日文词	💡 荒诞记忆密码（大白话）
activity	🔒 NTPサーバ数を表示	🔒 比赛很活跃（activity），过去**数一数（数）**人数。
sources	🔒 時刻のソースを表示	🔒 寻找水源（source），把水源列表列出来。
sourcestats	🔒 ソースの統計情報を表示	🔒 后缀带 stats（统计），秒选 統計情報。
tracking	🔒 同期の詳細情報を表示	🔒 一路追踪（tracking）到底，连根拔起查详细（詳細）。

19. タイムゾーンの情報がバイナリファイルとして格納されているディレクトリは/usr/share/zoneinfo/

システムのログ

1. 当你刚刚修改完系统日志服务（rsyslogd）的配置文件 /etc/rsyslog.conf 之后，身为合格的管理员，我们绝对不能坐在那里干等着真实的错误发生才去验证。我们必须**主动出击，手写一条假日志“钓鱼”执法**，看看系统能不能按预期把它分类并记录下来。

在 Linux 命令行中，专门用来由人类手写、自定义生成一条系统日志消息（システムログメッセージ）并发送给日志守护进程的工具是 **logger**。

可以在终端里直接敲，也经常被塞进 Shell 脚本（シェルスクリプト）里。比如当脚本运行出错时，可以用 **logger** 手动喷出一条“报告老大，备份失败！”的日志扔进系统的 /var/log/messages 里面。

Linux 日志服务（Syslog）核心三大件判定表

在 LPIC-1 的日志管理章节中，出题人最喜欢把下面这三个长得极其相似的“Log 家族成员”放在一块考你，请务必用“身份分工”大白话把它们理清：

名字	⚙️ 它的真实身份与职责	🔒 考场核心判定日文词
rsyslogd	后台的“日志总局局长”。常驻后台，负责按照 /etc/rsyslog.conf 的规定分配、记录、保存所有日志。	デーモン、ログを管理
logger (本题)	前台的“小报记者/线人”。负责由人类手动输入，或者在脚本里自动生成一条消息丢给局长。	手動で、メッセージを生成する
logrotate	收废品的“保洁阿姨”。负责定期把旧日志打包（如 .log.1、.log.2），把写满的日志清空轮替。	ログファイルのローテーション

看清 **-p** 选项的变形考法：这道题的题干里提到了“指定优先级（指定したプライリティ）”。**logger** 在配合测试时，通常会带上 **-p**（Priority）选项，后面接 设施名.级别名（例如 **-p auth.notice**）。如果以后题目考你：“**logger** 用哪个参数来指定日志的级别和设施？”，不用怀疑，直接选 **-p** 即可。

2. 「/etc/rsyslog.conf」はrsyslog（reliable-syslog）の設定ファイルです。

rsyslogはsyslog（ログメッセージを送受信する規格）の問題点を克服した次世代のsyslogとして、主要なディストリビューションで採用されています。

rsyslogで扱うログメッセージは、以下のように分類されます。

ファシリティ	説明
auth、authpriv	認証サービス
daemon	デーモン各種
kern	カーネル
lpr	印刷サービス
mail	メールサービス
syslog	syslogdデーモン
user	ユーザーアプリケーション
local0～7	独自に使用可

主なファシリティ（ログの送信元）

プライオリティ	説明
emerg	非常に危険
alert	緊急対応が必要な状態
crit	危険な状態
err	一般的なエラーメッセージ
warning	警告メッセージ
notice	通知メッセージ
info	情報メッセージ
debug	デバッグ時に有益な情報
none	メッセージを送らない

プライオリティ（監視レベル）

在 `/etc/rsyslog.conf` 文件中，每一行有效的日志分流规则（被称为 Selector 行）都必须严格遵循以下黄金公式：

Plaintext

代码块

```
1  ファシリティ.プライオリティ      アクション（出力先）
```

- 左边是条件：由「ファシリティ（设施/哪个功能生的日志）」和「プライオリティ（级别/多严重的事）」组成，中间用 `.`（点号）焊接。
- 右边是去向（Action）：指定这批日志应该被写入哪个文件、或者喷到哪个控制台终端。
- 多条件组合：如果想给同一个去向指定多组条件，中间必须用 `;`（分号）进行隔开。

正确答案（4つ選択）

- `./dev/console`
- `mail.err /var/log/maillog`
- `auth.* /var/log/secure`
- `*.info;mail.none;syslog.none /var/log/messages`

4 个正确选项的硬核分析

1. `./dev/console`

- 大白话翻译：左边的 `*.*` 代表“所有设施（左星）的、所有级别（右星）的日志”。右边是 `/dev/console`（控制台）。意思是“不管是啥日志，全给我实时喷在控制台屏幕上”。完美符合标准文法。

2. `mail.err /var/log/maillog`

- **大白话翻译**：左边是“邮件设施（`mail`）”的“错误级别（`err`）”。右边是标准的邮件日志路径。完全正确。

3. `auth.* /var/log/secure`

- **大白话翻译**：左边是“认证与安全设施（`auth`）”的“所有级别（`*`）”。右边是 Linux 标准存放登录、输密码记录的日志文件 `/var/log/secure`。完全正确。

4. `*.info;mail.none;syslog.none /var/log/messages`

- **大白话翻译**：这是考场上最复杂的高段位正确选项。它用 `;`（分号）把三组条件连在了一起：
 - 第一组：`*.info`（所有设施的 info 级别日志都记录）；
 - 第二组：`mail.none`（但是邮件日志一条也不要）；
 - 第三组：`syslog.none`（系统日志也一条也不要）。
- 把这些过滤后的结果，统一扔进大杂烩日志 `/var/log/messages`。这在实际生产环境中是最标准的写法，完全正确。

8大核心设施（ファシリティ）与 8大严重级别（プライオリティ）

ファシリティ（设施/日志来源）：标识这条日志是系统的哪个部门（功能）生出来的。

プライオリティ（级别/严重程度）：标识这件事情有多大（从低到高递增）。Linux 的默认规则是：只要指定了某个级别，系统就会抓取“该级别以及比它更高（更严重）的所有日志”。

第一阵营：8大设施（ファシリティ）“各行各业”谐音大戏 🎭

我们把这 8 个单词（`auth`、`daemon`、`kern`、`lpr`、`mail`、`syslog`、`user`、`local0~7`）串成一个“系统大院里的各路神仙”故事：

1. `auth`（认证）➡ 谐音：“奥特（曼）” 🦸
 - 画面：奥特曼在门口当保安，负责检查**认证（認証サービス）**和密码。
2. `daemon`（守护进程）➡ 谐音：“带萌（小宠物）” 🐶
 - 画面：后台常驻的各种各种守护进程（デーモン各種），每天都带着一只萌宠在辛勤后台加班。
3. `kern`（内核）➡ 谐音：“啃（骨头）” 🍖
 - 画面：系统的最核心——内核（カーネル），正在最底层默默地啃着硬骨头。
4. `lpr`（打印）➡ 谐音：“老骗（人）” 🖨️
 - 画面：公司那台**打印机（印刷サービス）天天卡纸，真是老骗人（`lpr`）**的破机器。
5. `mail`（邮件）➡ 直译：“买（东西）” 📦

- 画面：天天收到**邮件服务（メールサービス）发来的账单，提醒我又买（mail）**多了。

6. **syslog**（系统日志）➡ 直译：“死脑筋” 🧠

- 画面：日志服务自己（syslogdデーモン）极其死脑筋，一笔一画记录着自己的一切。

7. **user**（用户）➡ 直译：“普通人” 👤

- 画面：用户应用程序（ユーザーアプリケーション），就是给**普通用户（user）**用的。

8. **local0~7**（自定义）➡ 谐音：“老抠” 💰

- 画面：这群老抠门，舍不得用公共配置，非要独自留着（独自に使用可）8个兜（0~7）自己用。

第二阵营：8大级别（プライオリティ）“从低到高”生死逃生记 🚒

这8个级别在考场上最难的是“从低到高”的升级顺序（none ➡ debug ➡ info ➡ notice ➡ warning ➡ err ➡ crit ➡ alert ➡ emerg）。我们把它想象成一场“公司大楼着火”的灾难片进化史：

1. **none**（无）➡ “风平浪静”：大楼里什么事都没发生，不发任何消息（メッセージを送らない）。
2. **debug**（调试）➡ 谐音：“低八度” 🦋：程序员发现个小Bug，**低八度（debug）**地小声逼逼（デバッグ時に有益）。
3. **info**（信息）➡ 谐音：“应付” ⓘ：前台进了一条普通情报信息（情報メッセージ），大家嘴上应付，心里没当回事。
4. **notice**（通知）➡ 谐音：“漏通知” 📢：领导发了群公告通知（通知メッセージ），说大楼冒烟了，千万别漏通知（notice）。
5. **warning**（警告）➡ 谐音：“网宁（网不宁）” ⚠️：火势起来了，网络开始不宁静，响起了刺耳的警告（警告メッセージ）。
6. **err**（错误）➡ 谐音：“呃（完蛋）” ❌：火烧到脚了！大家心里**“呃（err）”的一声，大事不妙，爆出一般性错误（一般的なエラー）**。
7. **crit**（关键危机）➡ 谐音：“哭（天喊地）” 😭：火已经封门了！处于极度危险的状态（危険な状態），大家开始**哭（crit）**天喊地。
8. **alert**（紧急警报）➡ 谐音：“按铃（儿）” 🔔：消防员来了，疯狂按铃儿（alert），进入必须紧急应对的状态（緊急対応が必要）。
9. **emerg**（灾难崩溃）➡ 谐音：“伊美（救护车）” 🚑：全楼烧光，大楼崩塌！极其危险（非常に危険），赶紧叫**伊美（emerg / Emergency）**救护车拉人！

“包含性”高频大陷阱：如果在考场上遇到某行配置写着 `mail.warning`，题目问你：“以下哪种邮件日志不会被记录？”

- 秘籍：你要立刻在脑子里过一遍“大楼火灾进化史”。由于 `warning` 触发了“网不宁”，那么比它更严重的“呃（`err`）”、“哭（`crit`）”、“按铃（`alert`）”、“伊美（`emerg`）”通通都会被记录。而比它温柔的“漏通知（`notice`）”和“硬服（`info`）”则会被无情漏掉、不作记录。

锁定唯一复数 `auth`，`authpriv`：看清楚大纲第一行，只有“认证服务”是由两个单词组成的。如果在多选题里问你“哪些属于标准的设施（`ファシリティ`）”，只要看到长得很像的 `auth` 和 `authpriv`，这两个全都是正义的伙伴，大胆一起选上！

5. journalctl コマンド

オプション	説明
<code>-a (--all)</code>	デフォルトでは画面表示されない文字も含めて省略せず、すべて表示
<code>-b (--boot)</code>	特定のシステム起動時のログを表示する。 0は今回の起動時を意味し、ログの古いほうから1（最古のログ）、2…と指定するか、直近の起動時から-1（前回起動時）、-2（2回前の起動時）…と指定する
<code>-f (--follow)</code>	新規に追加されたログをリアルタイムに表示（ <code>tail -f</code> と同様）
<code>-k (--dmesg)</code>	カーネルからのメッセージを表示（ <code>dmesg</code> コマンドと同じ出力）
<code>-l (--full)</code>	画面表示可能なすべてのログを表示
<code>-n (--lines)</code>	直近のログから指定行数分を表示（行数未指定時のデフォルトは10行）
<code>--no-pager</code>	デフォルトで使われる <code>less</code> コマンドを使用しない
<code>-o (--output)</code>	ログの出力形式を指定 通常より詳細（ <code>verbose</code> ）な形式や、JSON（ <code>json</code> ）形式が指定できる。 ※JSON: JavaScript Object Notation
<code>-r (--reverse)</code>	最古のログからの表示（デフォルト）を逆にし、最新のログから表示
<code>--since</code>	指定した日付時刻以降のログを表示
<code>-u (--unit)</code>	特定のユニットからのログを表示
<code>--until</code>	指定した日付時刻までのログを表示

`journalctl` は用来查询由 `systemd-journald` 服务收集的二进制日志的工具。为了能从海量的日志中瞬间捞出我们需要的内容，它支持使用 `变量名=值` 的书式来进行过滤。

由系统或内核自动记录的、关于进程自身属性的变量（如进程ID、用户ID、组ID等），在 `systemd` 的底层规则中，变量名必须全部大写，且必须以单下划线（`_`）开头。

特定变量：指定进程ID（Process ID）的变量名是 `_PID`。

- 公式书式：`journalctl _PID=进程ID`

journalctl_PID=100

例题：systemdの動作するシステムで、sshd.service ユニットからのログのみを出力させたい。
正しいコマンドは

journalctl_SYSTEMD_UNIT=sshd.service

journalctl -u sshd.service

journalctl 命令用来查看由 systemd-journald 守护进程收集的二进制系统日志。因为是二进制日志，必须通过它带上不同的参数来过滤输出。

- **核心特性**：如果不加任何参数，默认会使用 less 命令的分页器（Pager）从旧到新展示所有日志。
- **常用选项**：包含限制行数、实时跟踪、查看内核、指定单元服务等多种精准维度。

我们把大纲中（image_ac90d4.png）最常考的 9 个核心参数 串成一个“特工潜入秘密基地”的荒诞故事：

- f (--follow)** → 谐音/直译：“放（f）肆跟踪 / Follow 关注” 🕵️
 - **荒诞画面**：特工潜入基地，放肆（-f）地在后面 Follow（跟踪）敌人的脚步，敌人走一步，日志在屏幕上就实时刷新滚动出一行。
 - **记忆链接**：-f = Follow，代表实时滚动显示最新追加的日志（实时查看）。
- n (--lines)** → 谐音/直译：“拿（n）几行 / Number 数字” 1 2 3 4
 - **荒诞画面**：特工时间紧迫，不能看全本，大喊：“快！只给我拿（-n）最新的 10 行情报出来！”
 - **记忆链接**：-n = Number of lines，显示最近指定行数的日志（不指定默认就是 10 行）。
- k (--dmesg)** → 谐音：“内核（k）/ 咔（k）的一声” 🧠
 - **荒诞画面**：特工的皮鞋踩到陷阱，咔（-k）的一声，直接惊动了大楼的最核心——内核（Kernel）！
 - **记忆链接**：-k = Kernel，只查看内核产生的日志消息（效果等同于 dmesg 命令）。
- u (--unit)** → 直译：“单元（Unit）/ 优（u）盘” 💾
 - **荒诞画面**：为了拷贝资料，特工拿出一个高级优（-u）盘，插入特定的 Service 单元（Unit）服务器里吸取日志。
 - **记忆链接**：-u = Unit，只查看某个特定 Systemd 服务单元（如 sshd）的日志。
- r (--reverse)** → 直译：“热（r）乎反转 / Reverse 倒车” ↺
 - **荒诞画面**：平时看日志都是先看10年前的旧账，特工嫌不热乎，一脚挂上 Reverse（倒挡 -r），让最新、最热（-r）乎的日志直接顶在屏幕最上面！
 - **记忆链接**：-r = Reverse，反转顺序，优先显示最新（最热乎）的日志。

6. `-b` (`--boot`) → 直译：“不 (b) 要以前 / Boot 开机” 🦶

- 荒诞画面：特工说：“以前的事情我不 (`-b`) 管，我只要看这次 **Boot (开机)** 启动以来的日志！”
- 记忆链接：`-b` = **Boot**，只显示本次（或指定某次）系统启动后的日志。

7. `--no-pager` → 直译：“NO! 不要配角 (Pager)” ❌

- 荒诞画面：平时看日志，系统总会找一个叫 `less` 的配角来搞分页。特工怒吼：“NO! 不要配角 (`--no-pager`) 打扰我，把日志一次性像瀑布一样全部喷在屏幕上！”
- 记忆链接：`--no-pager` = 不用 `less` 分页命令，直接全量打印输出。

8. `--since` 与 `--until` → 英文直译：“自从 → 直到” ⌚

- 荒诞画面：特工看着时间手表：“自从 (`--since`) 昨天，直到 (`--until`) 现在，这期间基地里发生了什么？”
- 记忆链接：`--since` 指定起始时间，`--until` 指定结束时间。

“`journalctl` 选项多，特工潜入听我说。”

“`-f` 跟踪实时波，`-n` 拿来十行多。”

“`-k` 咔嚓内核惊，`-u` 优盘单元清。”

“`-r` 倒车看最新，`no-pager` 一喷到底不用停！”

📝 考试小秘籍

- a. 组合拳联动考法 `-fn 5`：考场上特别喜欢考参数的合体。比如题干问你：“如何实时查看某个服务，并且一上来只显示最新的 5 行？”
- 秘籍：实时查看用 `-f`，限制行数用 `-n`。合体写成 `journalctl -f -n 5` 或者 `journalctl -fn 5`。看到这种组合选项大胆选它！
- b. 辨析 `-k` 与传统 `dmesg`：如果题目问：“哪个命令或者选项可以查看カーネル（内核）的日志？”
- 选项里如果有独立命令，选 `dmesg`；
 - 如果是问 `journalctl` 的参数，立刻想到“咔 (`-k`) 的一声惊动内核”，秒选 `-k`！这两个在考场上是完全等价的。
- c. 当管理员在终端里直接敲下单独的 `journalctl` 命令而不加任何后缀时，Linux 为了防止海量的日志像瀑布一样一瞬间刷满整个屏幕，系统会非常贴心地自动启动 `less` 软件（分页器/ページャ）。
- 默认情况下，`journalctl` 是“通过分页器 (less) 来展示日志”的（デフォルトではページャによる表示を行う）
 - 它的画面会停留在第一页，左下角出现一个冒号 (`:`)，允许你用上下方向键一行一行地、优雅地翻页阅读。



journalctlを使わなくてもlessコマンドでログが参照できる（不用journalctl，直接用less也能读日志文件）

- 死因：极其高频的致命陷阱！记住，systemd-journald 收集存放日志的物理文件，100%是硬核的“二进制格式（バイナリファイル）”，根本不是纯文本。如果你试图绕过命令，直接去后台 # less /var/log/journal/... 强行查看，你只会看到满屏幕的文字化け（乱码）。我们必须且只能通过 journalctl 翻译官才能看到正常的人类文字。

6. 这道题考查的是现代 Linux（systemd 时代）中，如何将任意普通命令的输出（Stdout/Stderr）直接管道对接到系统日志（Journal）的硬核命令。

它和我们前面学过的 logger 命令非常像，但 logger 主要是由人类手动输入一串文本字符串丢给日志系统；而本题的命令则是专门用来“包裹（Cat）”一个现成的 Linux 命令，把该命令跑出来的所有结果一股脑塞进二进制日志里。

在 systemd 接管系统后，除了传统的 logger 之外，系统提供了一个专门的工具叫 systemd-cat。

systemd-cat 可以作为一个管道，或者直接作为前缀，用来执行指定的命令，并将该命令的标准输出和错误输出全部重定向发送给 systemd-journald 守护进程。

官方书式：systemd-cat [オプション] [コマンド]

这里的 cat 借用了 Linux 传统命令 cat（连接/拼接）的概念。它的意思是“把命令的输出结果像小猫接水管一样，连接到二进制日志的大水库里”。

✎ 考试小秘籍

- “读写分离”秒杀律：考场上只要考到 systemd 日志，百分之百会把 journalctl 和 systemd-cat 放在同一个多选题或单选题里让你挑。请在心中默念：“ctl 是 Control（控制/查看），cat 是管道接水管（写入）。”只要题干问的是 書き込む（写入/生成），直接把 journalctl 判死刑划掉，秒选 systemd-cat。
- 简答填空题的拼写：LPIC-1 偶尔会出这道题的填空题。记住中间有一个 -（连字符），不要漏写。只要想到“系统级别的猫（systemd-cat）”，拼写就能稳稳拿下了。

7. 题目：journalctlコマンドでのログ出力において、前回の起動からシャットダウンまでのログを出力させたい。必要な条件と、正しいコマンドはどれか

✓ 正确答案（2つ選択）

- /var/log/journal ディレクトリが存在する
- journalctl -b -1

这道多选题非常经典，它把 journalctl 的命令参数考点与 Linux 底层的存储机制考点完美融合在了一起。

出题人问的是：如何输出“上一次开机（前回の起動）到这一次关机之间”的日志。要达成这个目的，我们不仅需要选对命令，还要满足一个“大前提条件”。

只要搞懂了“倒车数字”和“内存 vs 硬盘”的大白话逻辑，这道题的 2 个正确答案就能手到擒来。

💡 知识点总结

在默认情况下，现代 Linux 的二进制日志服务 `systemd-journald` 其实非常“抠门”，它天生默认把日志存在 **内存（RAM）** 里面。

- **必备条件（日志永久保存）**：保存在内存里的数据是“挥发性”的，也就是说**一旦关机或重启，内存里的旧日志就会被彻底洗掉**。如果我们想要跨越生死、在重新开机后还能看到“上一次开机时”的旧日志，就必须让系统把日志写进硬盘（永续保存）里。在 Linux 中，这个雷打不动的指定硬核硬盘路径就是 `/var/log/journal`。
- **正确命令（指定哪次开机）**：利用 `-b` (`--boot`) 参数可以指定查看哪一次开机周期的日志。
 - `0` 代表：**本次开机**（当前正在运行的这一次）。
 - `-1` 代表：**上一次开机**（也就是往回倒车 1 次）。

在 Linux 的世界里，`/run` 目录代表“跑步奔跑（内存数据），断电即断气”；而 `/var` 目录代表“蓄水池（硬盘数据），断电也还在”。`systemd-journald` 有一个非常聪明的自动化机制：只要它在系统里检测到 `/var/log/journal` 这个目录存在，它就会一拍大腿说：“懂了！管理员想让我把日志焊死在硬盘里！”从此开启永久保存模式。这样，即使重启了，上一次的日志也能被稳稳留住。

4. 在 `/etc/rsyslog.conf` 中，右边的去向（Action）不仅可以写本地文件路径（如 `/var/log/messages`），还可以写 `@` 符号加远程主机名，用来把日志通过网络源源不断地同步到专门的“中央日志服务器”上。

网络转发的双雄暗号：

- **单艾特** `@` ➡ 代表 **UDP 协议**（传输快、但不保证 100% 到达，适合大量普通日志）。
- **双艾特** `@@` ➡ 代表 **TCP 协议**（有三次握手、极其可靠，适合重要日志）。

```
*.warning @@test
```

- **大白话翻译**：所有设施的 `warning`（警告）级别以及比它更严重的日志，通过 **双** `@@`（**TCP 协议**）转发给外部服务器 `test`。
- 规则里写的是 `*.warning`，意思是“只要严重程度达到或超过 `warning`，就触发 TCP 转发”。

```
*.err @prod:514
```

- **大白话翻译**：所有设施的 `err`（错误）级别以及比它更严重的日志，通过 **单** `@`（**UDP 协议**）转发给外部服务器 `prod` 的 `514` 端口。

“网络 VS 文件”一秒去伪：只要在右边看到了 @ 符号，它后面的那个单词（比如 prod、test）在日文解释里必须被称为「外部サーバ（外部服务器）」或者「リモートホスト（远程主机）」。任何把它翻译成“文件（ファイル）”或者“终端控制台（コンソール）”的选项，全部都是语法盲选项，直接用铅笔划掉。

Linux 的日志过滤逻辑是“安全防御逻辑

在 Linux 日志（Syslog）中，配置文件里的级别（如 *.warning）指定的不是一个“固定的区间”，而是一个“触发报警的门槛（阈值）”。Linux 的规则是：“只要事情的严重程度『达到』或者『超过』了我指定的门槛，就必须记录下来。”

当大楼都炸了（emerg）的时候，身为安全总监的你，总不能跑去跟老板说：“报告老板，因为我之前只让警报器在有【警告 warning】的时候响，现在大楼是【灾难 emerg】，比警告高级多了，所以警报器就保持沉默没有响。”这显然是极其荒谬的。在安全领域，连“警告”这种小事我都要记，那么比警告更严重的“错误（err）”、“危机（crit）”、“大楼炸了（emerg）”，我就更不可能放过了，必须无条件通通记下来！

这就是为什么写着 *.warning 时，系统会“往上（往更严重的方向）”抓取全部日志的真正原因。

6. systemd-journald デーモンの設定ファイルは/etc/systemd/journald.conf

systemdの動作するシステムではsystemd-journaldデーモンを動作させ、ログの一元管理を行います。systemd-journaldはsystemdから起動したプロセスの標準出力やsyslogへのログメッセージをバイナリ形式で記録します。

7. systemdの動作するシステムで、各Unitが出力する表示可能なログをすべて表示させたい。指定すべきオプションは-l 和 --full

-l (--full)	画面表示可能なすべてのログを表示
-------------	------------------

8. 「/etc/rsyslog.conf」ファイルを以下のように設定している。この設定の説明として正しいものはどれか。(3つ選択)

在 /etc/rsyslog.conf 中，每一行规则都在做“分流”。这道题的题干给出了 3 条经典的转发规则，它们各自代表了一个极其高频的考场方向：

a. 第一行：authpriv.* /var/log/secure

- 考查设施拼写：authpriv 代表“认证与机密安全服务”。后面是 .*，说明无论大小事，只要是认证生的日志，全存进右边的安全日志文件里。

b. 第二行：*.warning @server1

- 考查网络转发 + 包含性：右边是单艾特 @，代表利用 UDP 协议转发给外部服务器 server1。因为是 *.warning，所以根据火灾警报器铁律，警告（warning）以及比它更严重的日志都会被转发。

c. 第三行: `kern.err *`

- 考查全新右侧符号 `*`: 左边是内核 (`kern`) 发生错误 (`err`) 及以上日志。右边空降了一个孤零零的 `*` (星号), 在 Linux 日志语法里, 它不是文件, 而是代表“大喇叭全网广播”——强行弹窗显示在当前系统里所有正在登录作业的用户们的控制台(屏幕)上。

✓ 正确答案 (3つ選択)

- 認証サービスに関する全てのプライオリティのログを「/var/log/secure」に出力する (第1項)
- 全てのファシリティの警告以上のログをホストserver1のsyslogに送信する (第5項)
- カーネルに関するエラー以上のログを全てのユーザのコンソールに出力する (第6項)

🔴 3个正确选项的硬核分析

- 第1项: 認証サービスに関する全ての... (将认证服务的所有级别日志输出到...)
 - 大白话: 完美对应第1行 `authpriv.*`。`authpriv` 在日文题库里对应的翻译就是“認証サービス”, `*` 代表所有级别。完全正确, 秒选。
- 第5项: 全てのファシリティの警告以上のログをホストserver1の... (将所有设施的警告以上日志发送给主机 server1...)
 - 大白话: 完美对应第2行 `*.warning @server1`。
 - 左边的 `*` = 全部的设施 (全てのファシリティ)。
 - 中间的 `warning` = 触发门槛是警告, 因为有包含性, 所以日文描述极其精准地用了警告以上。
 - 右边的 `@server1` = 单艾特, 发送给外部主机 (ホストserver1)。完全符合铁律, 秒选。
- 第6项: カーネルに関するエラー以上のログを全てのユーザの... (将内核错误以上的日志输出到所有用户的控制台)
 - 大白话: 完美对应第3行 `kern.err *`。
 - 左边 `kern` = 内核 (カーネル)。
 - 中间 `err` = 触发门槛是错误, 日文描述再次精准使用了 エラー以上。
 - 右边单星号 `*` = 所有用户的控制台 (全てのユーザのコンソール)。三个维度全部对齐, 秒选。

“左边条件右边路, 看到【@】是网络转外部。”

“题干只要写级别, 日文必选【～以上】才符合。”

“右边要是空降一个【*】(单星号), 不要误认成文件路径;”

“那是大喇叭全网广播, 【所有登录用户】的屏幕全部强行弹窗!”

9. 次世代の syslog として開発されたものは **syslog-ng** 和 **rsyslog**

在现代 Linux 系统的发展过程中，为了解决老旧 **syslog** 的痛点，诞生了两个最著名的**下一代增强版日志服务**。它们完美继承并极大扩展了传统配置，成为了现代各 Linux 发行版的标配：

rsyslog：目前 **CentOS / RHEL / Ubuntu** 等绝大多数主流发行版的**默认（默认）内核级日志系统**。它的名字意为 **rocket-fast syslog**（火箭般神速的 syslog），最大的特点是支持多线程、高性能以及极其丰富的插件和数据库写入功能。

syslog-ng：名字里的 **ng** 代表 **Next Generation**（下一代）。它是最早提出模块化、强大过滤和支持 TCP 传输的次世代日志服务，在很多企业级定制系统和安全领域（如 SUSE Linux）被广泛采用。

10. systemdの動作するシステムで、ユニットが出力するログを扱うデーモンプログラムは **systemd-journald**

在 systemd 接管的现代 Linux 系统中，负责收集内核、系统服务、以及各种ユニット（Unit）输出的日志的核心程序是 **systemd-journald**。所有服务的标准输出（Stdout）和标准错误（Stderr）都会自动被它接住，并安全地写进二进制日志文件中。

反例：**rsyslogd**虽然它也是一个デーモン，但它是负责将日志进行网络转发或写入传统文本文件（如 `/var/log/messages`）的**次世代 syslog 服务**。它现在不再直接向各个单元（Unit）捞取一手日志，而是从 **systemd-journald** 那里“二手对接”过来。

11. ログローテーションの設定ファイルは「`/etc/logrotate.conf`」で、実行するには「logrotate」コマンドを使用します。通常は「cron」によって定期的に実行されます。

系统每天都在产生海量的日志，如果不定期进行裁剪和归档，硬盘迟早会被撑爆。而负责这份“保洁工作”的工具就是 **logrotate**，只要找到它对应的配置文件，就能自由更改日志保留的天数和轮替规则。

“`/etc` VS `/var`” 斩杀律：考场上只要题目问的是“**更新/修改某种功能（設定を変更・更新したい）**”，凡是选项开头是 `/var/log/...` 的，一律直接打大叉（✗）。因为 `/var` 是放动态数据的，改里面的日志内容根本无法改变系统规则。正确答案必定在 `/etc` 开头的选项中产生。

12. 在 systemd 接管系统后，我们使用 **systemctl** 命令来控制各种服务的生命周期。

- **核心考点**：当配置文件发生根本性修改时，让服务应用更改最稳妥、最通用的方法是「**重启服务（Restart）**」。
- **标准命令**：`systemctl restart 服务名`
- **完整书式**：`systemctl restart rsyslog`

restart 的本质是“先关闭服务（Stop），再瞬间开启服务（Start）”。在关闭和重新开启的交替瞬间，**rsyslogd** 进程会重新去扫描并加载 `/etc/rsyslog.conf` 里的每一行新规则。这是我们在考场上遇到“修改配置后如何应用更改（変更を適用するには）”时的万能保底解。

其实除了 **restart** 之外，还可以用 `systemctl reload rsyslog`（只重新加载配置而不重启进程）

13. メールサービスの全てのプライオリティのログを「/var/log/maillog」に出力したいが、大量にログが発生してもディスクへの書き込み負荷を上げたくない。「/etc/rsyslog.conf」でどのように設定すればよいか。

💡 知识点总结

在 `/etc/rsyslog.conf` 中，右边的输出路径如果以 `-`（减号）开头，代表“禁用同步写入（非同期書き込み / 异步刷盘）”。

- 核心考点：
 - 默认状态（不加减号）：Linux 每产生一条日志，系统就会硬核地立刻“死磕”硬盘，必须等数据实实在在写进磁盘后，才允许进行下一步。这叫**同步写入**。如果邮件服务（Mail）遭遇大规模并发，磁盘的 I/O 写入负载会瞬间飙升到 100%，导致整台服务器卡死。
 - 加减号状态（`-/path`）：告诉系统：“先把日志暂存到内存的缓冲区（Buffer）里，等攒够了一大波，或者磁盘闲下来的时候，再批量一次性写入硬盘。”这叫**非同步写入**。
- 特定功能：专门用来降低高并发下的磁盘写入负载（書き込み負荷を下げたい）。

✅ 正确答案

`mail. -/var/log/maillog*`

“邮件日志像轰炸，疯狂写盘磁盘挂。”

“路径前面加【`-`】（减号），那是给磁盘【减减压】。”

“内存先当缓冲垫，异步刷盘低负载。”

“考场看到【`-/var`】，保护硬盘选它对！”

14. `logrotate`（日志轮替）本质上就是一个**定时的自动化保洁阿姨**。因为系统日志天天都在疯狂增加，如果不做限制，硬盘总有一天会爆炸。阿姨的工作就是：**定期把老日志打包、改名、压缩，把太久远的扔掉，并换上一个干净的新本子继续记。**

- a. ログローテーションの設定ファイルは「/etc/logrotate.conf」で、実行するには「logrotate」コマンドを使用します。通常は「cron」によって定期的に行われます

項目	説明
<code>compress</code>	切り替え後のログを圧縮する
<code>create</code>	ログ切り替え時に新たな空のログファイルを作成する
<code>daily, weekly, monthly, yearly</code>	ログの切り替えタイミング（日次、週次、月次、年次）
<code>include</code> ディレクトリ名	指定したディレクトリ配下の設定ファイルを読み込む
<code>mail</code> メールアドレス	ログ切り替えによって削除されるログの内容を指定されたメールアドレスにメールする
<code>postrotate ~ endscript</code>	ログ切り替え後に実行するスクリプト
<code>prerotate ~ endscript</code>	ログ切り替え前に実行するスクリプト
<code>rotate</code>	切り替え後に保持する世代数

阿姨的“保洁工作流程”

我们顺着阿姨的工作时间线，把这 8 个硬核指令分为四组，你会发现记忆起来极其顺畅：

第一组：决定什么时候开始打扫（时间周期）

- `daily, weekly, monthly, yearly`
 - 日文定义：ログの切り替えタイミング（日次、週次、月次、年次）
 - 大白话：阿姨的“定闹钟服务”。决定是每天（daily）、每周（weekly）、每月（monthly）还是每年（yearly）过来彻底打扫一次。记住，它只决定“什么时候开始切分”，不决定怎么删。

第二组：打扫时的具体动作（核心处理）

- `create`
 - 日文定义：ログ切り替え時に新たな空のログファイルを作成する
 - 大白话：“原地换新本子”。阿姨把写满的旧日志文件拿走后，必须在原地立马新建一个干净的空文件，让系统服务能够无缝继续往里写日志。
- `compress`
 - 日文定义：切り替え後のログを圧縮する
 - 大白话：“抽气真空袋”。旧的纯文本日志太占地方，阿姨用 `gzip` 工具给它“吸成真空包”（变成 `.gz` 结尾），极大地节省硬盘空间。
- `rotate`
 - 日文定义：切り替え後に保持する世代数
 - 大白话：“储物柜最大格数”。后面通常接数字（如 `rotate 4`），代表储物柜里最多只放 4 代老祖宗。当第 5 代由于时间到了要被塞进来时，最老的内代就会被淘汰丢掉。

第三组：前置和后置的“花式操作”（脚本与收尾）

- `prerotate ~ endscrip`
 - 日文定义：ログ切り替え前に実行するスクリプト
 - 大白话：“打扫前的准备”。在切分之前（Pre-）要执行的特殊系统命令。
- `postrotate ~ endscrip`
 - 日文定义：ログ切り替え後に実行するスクリプト
 - 大白话：“打扫后的收尾”。在切分之后（Post-）要执行的命令。比如告诉服务：“日志已经换新本子了，请你重新把新本子读起来（经常配合 `systemctl reload` 使用）”。
- `mail メールアドレス`
 - 日文定义：ログ切り替えによって削除されるログの内容を指定されたメールアドレスにメールする

- **大白话**：“寄回老家寄生”。有些日志快要超过 `rotate` 的代数被删掉了，但非常重要。阿姨在把它毁尸灭迹之前，最后一次把它当成邮件附件**发送给管理员的邮箱**，作为离线归档。

第四组：高效率分流管理（加载配置）

- `include` ディレクトリ名

- **日文定义**：指定したディレクトリ配下の設定ファイルを読み込む
- **大白话**：“召唤分身小弟”。主配置文件 `/etc/logrotate.conf` 不想把所有服务的规则都写在自己肚子里。用 `include /etc/logrotate.d` 就能一键把这个小包间目录下的所有独立小配置全都吸纳进来一起执行。

“【`Pre-`】就是【前（Before）】，【`Post-`】就是【后（After）】，`~ endscript` 包在两头走。”

“【`rotate`】看【世代】，【`compress`】负责【压】。”

“【`create`】造【空文件】，【`include`】去【读配置】。”

“【`weekly`】只是触发点，绝不是当场做【削除】！”

考试小秘籍

- a. “时间不等于删除”是万年不变的第一大坑：再强调一遍，`daily/weekly/monthly/yearly` 对应的正确说明里，绝对不能出现“删除（削除）”字样。看到“毎週ログを削除する”这种选项直接打大叉（✗）。
- b. `postrotate` 的常客命令：考场上如果考你配置文件的连线题或者看代码多选题，你经常会在 `postrotate` 和 `endscript` 之间看到这样一行字：

代码块

```
1 /usr/bin/systemctl reload rsyslog > /dev/null 2>&1
```

- **秘籍**：不要被这行长代码吓到。这正是我们上一题学的“应用新更改”！因为阿姨用 `create` 换了一个新本子，必须在 `postrotate`（切分后）大喊一声：“rsyslog！我换新本子了，你重新加载（`reload`）一下，往新本子里写！”完美形成考点闭环。

15. systemdの動作するシステムで、各Unitが出力するログを参照したい。使用するコマンドは **journalctl**

- `systemctl`：systemdを管理するコマンドのため誤りです。
- `cat` • `less`

systemd-journaldの記録するログはバイナリ形式であり、`cat`や`less`では正常に表示できないため、誤りです。

メール配送エージェント(MTA)の基本

1. 这道题考查的是 Linux 网络服务中，代表性的 MTA（邮件传输代理 / メール送信サーバ）的各自特点。

题目中提到的“保持与 sendmail 的高兼容性”、“安全性与速度提升”以及“现代很多发行版的默认 MTA”，是某款明星级邮件软件最标签化的特征。

代表的なMTAのうち、次の説明に該当するものを選びなさい。説明) sendmailとの高い互換性を保ちながら、処理速度を上げ、セキュリティも向上させている。設定が平易。最近の一部のLinuxディストリビューションの標準MTAにもなっている。

在 Linux 发展史中，最早的邮件老大哥是 `sendmail`，但它因为配置复杂且频繁爆发安全漏洞而逐渐失宠。于是，后起之秀们纷纷登场：

- **Postfix**：由 IBM 倡导开发，核心设计理念就是**替代 sendmail**。它特意保留了与 sendmail 几乎一样的外部命令接口（如 `sendmail` 命令），但内部采用了多进程模块化架构，**处理速度极快，安全性极高，且配置文件（`main.cf`）非常平易近人**。它是目前大部分现代 Linux 发行版（如 RHEL、Ubuntu 等）默认的 MTA。

为什么选择 `postfix`？

- a. **高い互換性**：它完美兼容 sendmail 的命令行参数和别名机制。
- b. **セキュリティ向上**：它放弃了 sendmail 危险的“单进程 root 权限”设计，改为多进程、低权限运行。
- c. **標準MTA**：它已经是当今 Linux 世界无可争议的邮件服务器标准配置。

另外：

`exim`：它也是一种流行的 MTA（Debian 发行版曾默认使用它），但它的配置风格和设计理念与 `sendmail` 截然不同，它的配置语法非常庞大复杂，不符合“设置平易”的特点。但一样是当下标准MTA

名称	特徴
Postfix	sendmailとの高い互換性を保ちながら、処理を高速化している。設定が平易でsendmailよりもセキュリティが向上しているため、最近の一部のLinuxディストリビューションで標準のMTAになっている。
exim	Debian系の標準MTA。設定は平易だが、日本語の情報は比較的少ない。設定ファイルの内容はバージョンによって異なる。

Postfix：它的关键词是「**sendmailとの高い互換性**」（高兼容性）、「**処理を高速化**」（处理快）、「**セキュリティ向上**」（更安全）以及「**標準のMTA**」（现代主流标准）。

exim：它的关键词是「**Debian系の標準MTA**」（Debian默认）、以及「**日本語の情報は比較的少ない**」（日语资料少）。

邮件服务怎么选，看清标签不绕弯：安全高速兼容王，现代标准选 Postfix！Debian 专属资料少，冷门特殊是 exim！

2. 在 Linux 邮件服务器中，`/etc/aliases` 文件用于定义本地用户邮件的别名和转发规则。
- 基本语法格式：`国内收件人：转发目标1，转发目标2`
 - 冒号（`:`）的作用：它作为标准的字段分隔符。左边写**本地接收邮件的用户名**（如 `staff`），右边写**真正的投递目的地**（可以是一个或多个本地用户，也可以是外部的邮件地址，中间用逗号分隔）。

根据 `/etc/aliases` 的语法规则，这一行正确的写法应该是：`staff: user1, test@example.com`

当有人向 `staff@你的服务器.com` 发送邮件时，Postfix 看到这个冒号，就会自动把邮件复制两份，分别转发给本地的 `user1` 以及外部的 `test@example.com`。

注意！！

|（管道符）的连带考点：虽然 `|` 不能放在空栏的位置，但它在 `/etc/aliases` 的**右边字段**里大有作为。如果你想把收到的邮件不发给某个人，而是直接**丢给一个脚本程序去处理**，可以写成 `staff: "| /path/to/script.sh"`。考试中这也是一个非常高频的陷阱。

 考试小秘籍

- a. **致命连带命令 newaliases（必考）**：在修改完 `/etc/aliases` 文本文件后，Postfix 并不会立刻生效。因为为了提高读取速度，系统需要把这个文本编译成数据库文件（`aliases.db`）。你必须在终端手动敲一下 `newaliases` 命令（或者 `sendmail -bi`），配置才会真正生效！这是连带考题中最喜欢考的填空题。
- b. **符号区分**：不要把这个冒号和定时任务 `crontab` 的空格混淆。
3. メールの転送を行う方法には、以下の2つがあります。

メールの転送設定

設定ファイル	影響範囲	内容	設定の反映方法
<code>/etc/aliases</code>	システム全体	別名(エイリアス)と転送先を記載	「newaliases」コマンドを実行
<code>~/forward</code>	ユーザ個別	転送先を記載	即時反映

这道题考查的是 **用户自定义（一般ユーザ用）** 的邮件转发配置文件。

上一道题我们学过只有超级管理员（root）才能修改的全局别名文件 `/etc/aliases`。而当一个**普通用户**想要自主转发自己的邮件时，系统为他们提供了一个无需通过管理员就能配置的“自助通道”。

在 Linux 邮件系统中，普通用户可以通过在自己的家目录（ホームディレクトリ）下创建一个隐藏的文本文件来实现邮件转发。

- `~/.forward`：这就是专门供普通用户使用的邮件转发配置文件。
- 路径与命名规范：
 - `~`（波浪号）：代表当前用户的家目录（如 `/home/user1/`）。
 - `.`（点号）：代表它是一个隐藏文件（隠しファイル）。
 - `forward`：直译就是“转发”。
- 即时生效：与 `/etc/aliases` 修改后必须运行 `newaliases` 编译不同，`~/.forward` 只要保存，不需要任何额外命令，立刻就能生效！

当本地邮件服务器收到一封给 `alex` 的信时，它会先去 `alex` 的家目录下瞅一眼，看看有没有 `.forward` 这个隐藏文件。如果有，它就会读取里面的内容（比如写着 `test@example.com`），然后直接把信打包转寄过去。

安全权限（高频连带隐形考点）：因为 `~/.forward` 决定了邮件的去向，如果别人能随意修改你的 `.forward`，就能偷走你的所有邮件。所以为了安全，Linux 要求这个文件的权限必须非常严格——除了用户自己，绝对不能给其他任何人（Group 和 Others）开放“写入（w）”权限。如果权限过大（比如设成了 777），Postfix 会为了安全直接无视这个文件。

4. メールの送信や、受信メールを表示するコマンドは「mail」です。

書式：mail [-s 件名] [宛先メールアドレス もしくは ユーザ名]

オプションを付けずに「mail」コマンドを実行すると、メールプール「/var/spool/mail/」に格納されている受信メールを表示します。

以下は表示例です。

■件名「testmail」を宛先「`hoge@hoge.com`」へ送信する。

```
# mail -s testmail hoge@hoge.com
Hello!hoge.      (←メールの内容を入力)
.                (←先頭にピリオド「.」を入力して終了)
Cc:              (←Ccも続けて指定可)
```

5. 一般的なMTAにおいて、送信待ちのメールキューを一覧表示できるコマンドはmailq

在现代主流的邮件服务器（如 **Postfix**）中，系统提供了两个完全等价的命令来查看当前由于各种原因滞留、堵塞在系统里的邮件队列：

- **mailq 命令**：这是最传统、最通用的命令（源自老大哥 `sendmail`，但 Postfix 完美兼容它）。它的作用是列出当前在邮件队列中等待发送的所有邮件（メールキューの表示）。
- **postqueue -p 命令**：这是 Postfix 独有的原生管理命令（print queue），其输出结果和功能与 `mailq` 完全一模一样。

🔗 连带高频考点补充（队列清理）

考试除了考“怎么看（`mailq`）”，还非常喜欢考“怎么重发”和“怎么删除”：

- a. **强制重发 (Flush Queue)**：如果网络恢复了，你想让卡住的邮件立刻发出，可以使用 `postqueue -f`（或者 `postfix flush`）。
- b. **彻底删除卡住的邮件 (Delete Queue)**：如果发现有人在恶意群发垃圾邮件导致塞车，想要把队列里的脏邮件全删掉，使用 `postsuper -d ALL` 命令 (`delete ALL`)。

邮件堵塞发不出，快用 mailq 查水表。Postfix 原生也能看，postqueue 加小 p (`-p`) 效果好！

要想立马强行发，换成小 f (`-f`) 冲一波！

- 6. 这道题考查的是网络协议中最基础、也最核心的“邮件搬运工”协议。只要提到 MTA（邮件传输代理）在服务器之间**发送、转交和传输邮件**，必定使用的是同一个标准网络协议。（• MTA（Mail Transfer Agent）：メールの転送を行うプログラム。ネットワーク経由、またはサーバ内部（ローカル）のメール転送依頼に基づき、指定された宛先へメールを転送する。）

在邮件系统的生命周期中，不同的网络协议各司其职。

- **SMTP（简单邮件传输协议）**：它是整个互联网邮件传输的**核心骨干协议**。不管是客户端把邮件推送到服务器，还是两个邮件服务器（MTA）之间互相转发和投递邮件，**只要涉及到“发信”和“转信”，通通由 SMTP 独占**。
- 它的标准端口号是 `25`（非加密加密传输）以及 `465/587`（加密/提交端口）。

网络协议各不同，邮件流向看清冲：发信、转信（MTA）靠 SMTP，端口 25 跑得快！

收信、取信换赛道，POP3、IMAP 把信摘！

考试小秘籍

- a. **高频陷阱词**：在 LPI 考试里，只要题目问的是「**MTA 間の転送**」（MTA 之间的转发）或者「**メールの送信**」（邮件的发送），不用犹豫，闭着眼睛直接选 `SMTP`。
- b. **端口号联动**：经常会出填空题或选择题，考你这些协议对应的默认端口号：
 - `SMTP` $\rightarrow$ `25`
 - `POP3` $\rightarrow$ `110`
 - `IMAP` $\rightarrow$ `143`